设计文档

参考编译器介绍

参考项目为往届学长设计的编译器 DeNeRATe-cool/BUAA-2025-Compiler，采用 Java 语言实现。其词法分析通过扫描源程序识别不同类别的单词，并记录相应信息，为后续编译处理提供基础。

通过阅读参考源码，了解了词法分析的整体流程，以及单词识别、信息保存和结果输出之间的关系，并结合本项目的需求进行了设计。

编译器总体设计

总体结构

编译器采用 Java 语言实现，以 MIPS 汇编为目标代码，按编译阶段划分模块，通过增量开发逐步扩展功能。

词法分析部分将文件读写、单词识别和信息保存分开组织。程序入口读取源文件并调用词法分析器，分析器从左到右扫描源程序，将识别出的单词保存为有序列表，供结果输出及后续处理使用。

语法分析接收单词列表，按照文法识别程序结构，在建立语法树的同时记录输出。程序入口保存语法树根节点，统一收集词法和语法错误，供结果输出及后续阶段使用。

组成	主要职责
`Compiler`	读取源文件、调用分析器并输出结果
`LexerAnalyzer`	扫描字符，识别单词并记录行号
`Token`	保存一个单词的类别、原始内容和行号
`TokenType`	用枚举统一定义单词类别
`Parser`	启动语法分析，保存语法树根节点
`PaserContext`	集中保存语法分析的读取状态和输出记录
`GrammarJudge`	判断当前位置应选择的语法分支
`ParseRules`	按文法解析并构建语法树
`AstNode`	保存语法树节点及父子关系

接口设计

词法分析器接收完整的源程序文本，返回单词列表。每个单词保存类别、原始内容和行号，既用于输出识别结果，也为错误定位提供依据。

入口与分析器通过以下方式衔接：

```
LexerAnalyzer analyzer = new LexerAnalyzer(source);
List<Token> tokens = analyzer.analyze();
```

程序入口统一负责输入输出，将正常单词按“类别码、空格、原始文本”的格式写入 `lexer.txt`，将错误行号及类别码写入 `error.txt`，避免文件操作与扫描逻辑混杂。

语法分析直接使用内存中的单词列表。分析结果和语法树根节点分别通过以下接口取得：

```
Parser parser = new Parser(tokens);
List<String> output = parser.analyze();
AstNode root = parser.getRoot();
```

语法输出写入 `parser.txt`。语法树中的错误与词法错误合并，按行号从小到大写入 `error.txt`。

文件组织

项目根目录放置程序入口和配置文件，词法分析相关类集中在 `lexer` 包中，语法分析相关类集中在 `parser` 包中。测试程序及其输入输出集中保存在 `tests` 目录中。

```
SysY_Compiler/
├── Compiler.java
├── config.json
├── testfile.txt
├── lexer/
|   ├── LexerAnalyzer.java
|   ├── Token.java
|   └── TokenType.java
├── parser/
|   ├── Parser.java
|   ├── PaserContext.java
|   ├── GrammarJudge.java
|   ├── ParseRules.java
|   └── AstNode.java
└── tests/
    └── grammar/
        ├── testfile1.txt ~ testfile6.txt
        ├── input1.txt ~ input6.txt
        └── output1.txt ~ output6.txt
```

1. 词法分析设计

设计思路

词法分析采用手工编写的顺序扫描方式。源文件一次性读入后，根据当前字符的特征选择相应的识别规则，确定单词边界和类别，并记录原始内容与所在行号。

单词类别使用枚举表示，识别结果按顺序保存为 Token 列表。初始设计中，Token 的核心字段如下：

```
private final TokenType type;  // 单词类别
private final String content; // 原始内容
private final int lineNum;    // 所在行号
```

扫描时维护当前位置和行号，当前位置始终指向下一个尚未处理的字符。不同类别采用对应的识别方式：

- **标识符和关键字**：先读取完整名称，再判断是否为关键字，避免将带有关键字前缀的标识符拆开。

- **整数常量**：连续读取数字，保留原始文本。正负号单独作为运算符处理。
- **运算符和分隔符**：结合预读区分单字符和双字符运算符，括号、分号等分隔符直接识别。
- **字符和字符串常量**：按对应引号确定边界，保留引号及转义文本，转义引号作为常量内容处理。
- **空白与换行**：空白字符直接跳过，换行时更新行号，使单词和错误信息能够对应到源程序位置。

初始设计将单独出现的 `&`、`|` 统一标记为 `WRONGOP`，输出错误码 `a`，并继续扫描后续内容。单词识别与结果输出分开处理，正常单词和错误信息分别写入对应文件。

编码完成之后的修改

1. 统一扫描位置的含义

早期在读取完整单词后进行了多余的回退，可能导致单词截取不完整或重复扫描。修改后，扫描位置始终指向下一个尚未处理的字符，读取结束后直接保存完整单词。

截取单词时采用左闭右开的范围，结束位置保留在下一个未处理字符处，无需额外回退。

2. 调整运算符识别方式

最初连续读取相邻的运算符字符，容易将赋值号和负号等独立符号合并。修改后通过预读判断能否组成合法的双字符运算符，不能组合时分别识别。

例如，`a=-1` 中的 `=` 和 `-` 应分别识别，而 `<=`、`!=` 等则整体识别。

3. 补充转义引号处理

最初遇到引号就结束读取，导致转义引号被误认为结束标志，剩余内容还可能引发超时。修改后将反斜杠与后一个字符一起读取，使转义内容能够正确保留。

4. 拆分错误运算符类别

将统一的 `WRONGOP` 拆分为 `WRONGOPAND` 和 `WRONGOPOR`，分别表示单独出现的 `&` 和 `|`。记录错误后，将它们恢复为逻辑与、逻辑或的类别，供后续处理使用。恢复只改变类别，保留原始内容和行号，对外仍统一报告错误码 `a`。

为支持这一调整，将 Token 的类别字段改为可修改，并增加类型修改接口。以单独的 `&` 为例，处理方式为：

```
if (token.getType() == TokenType.WRONGOPAND) {
    errors.add(token.getNum() + " a");
    token.setType(TokenType.AND);
}
```

单独的 `|` 采用相同方式处理，记录错误后恢复为 `OR`。

大模型辅助代码评价与人工调整

大模型辅助用于理解单词信息的组织方式、检查扫描位置，以及定位转义引号造成的问题。其转义处理建议有助于避免常量提前结束；对于超出作业错误范围的检查建议，则结合任务要求进行取舍。错误运算符的分类和恢复方式也结合实际需求作了调整。

2. 语法分析设计

设计思路

1. 递归下降与模块划分

语法分析采用递归下降方式，从编译单元开始，按照文法依次识别声明、函数、语句和表达式。每个非终结符对应一个解析方法：必需部分按顺序处理，可选部分通过判断决定是否进入，重复部分通过循环处理。

判断方法与解析方法分别组织在 `GrammarJudge` 和 `ParseRules` 中。前者结合语法成分的可能开头及必要的预读选择分支，后者实际读取单词并建立节点。两者通过 `PaserContext` 共享读取位置和输出记录，避免反复传递分析状态。

例如当分析 `CompUnit` 时，根据文法，循环递归分析 `Decl` ， `FuncDef` 以及 `MainFuncDef` 部分。

2. 语法树组织

语法树采用通用的 `AstNode` 节点类，通过节点类型区分不同语法成分。非终结符节点保存结构名称，终结符节点保存具体 Token，各节点通过父子关系连接。核心结构为：

```
public String type;
public Token token;
public List<AstNode> children = new ArrayList<>();
public AstNode father;
```

解析方法返回对应的节点，由外层方法挂接到父节点下，逐步形成整棵树。树中保留括号、分号等终结符，既保存程序结构，也为错误定位提供信息。根节点由 `Parser` 保存，入口程序取得后可继续用于错误收集和后续分析。

3. 表达式与分支选择

表达式按运算优先级逐层解析。对于加减、乘除等左递归规则，先解析一个基本项，再通过循环读取后续运算符和操作数，避免直接递归调用自身。运算符和操作数按原顺序保存在同一层节点下。

对于开头相同的结构，结合所在位置和预读结果进行区分。例如，程序最外层的变量声明和函数定义通过名字后是否出现左小括号区分；函数体内则直接根据类型或声明关键字识别声明。

赋值语句和表达式语句先统一解析表达式，若随后遇到 `=`，再从已建好的表达式子树中提取左值，按赋值语句继续处理。同时删除多余的外层表达式输出，保留左值及数组下标内部的内容。

4. 输出与错误处理

终结符由统一的匹配方法检查、记录输出并推进位置；非终结符在完成解析后记录名称。

缺失分号、右小括号或右中括号的恢复分支采用补充虚拟节点的方式，分别标记错误码 `i`、`j`、`k`，行号取前一个已读取单词的行号。补节点时不移动当前位置，让当前单词留给后续结构处理。以缺少分号为例：

```
int line = tokens.get(index - 1).getNum();
Token missing = new Token(TokenType.SEMICN, ";", line);
AstNode iErrorNode = new AstNode(missing);
iErrorNode.isIError = true;
```

虚拟节点加入当前语法节点后，入口程序通过遍历语法树收集错误，再与词法错误统一排序输出。

编码完成之后的修改

1. 分离判断、解析和共享状态

最初将读取状态及辅助操作放在分析器中。随着解析方法增多，将判断方法和解析方法分别集中到独立文件，并用共享上下文保存单词列表、当前位置和输出记录，使各部分使用同一套状态。

2. 调整表达式标签的输出位置

课程组提供的部分文法存在左递归的情况，仅在整个表达式结束时输出标签不能符合题目要求。修改为在读取下一组运算符和操作数前先输出当前层标签，循环结束后再输出一次，保持规定的输出顺序。

3. 同步撤销试探解析的结果

处理缺少分号的 `return` 时，下一条赋值语句的开头可能被误当作返回表达式。修改后，在试探解析前保存读取位置和输出长度；若解析后遇到赋值号，则撤销试探子树，恢复读取位置，并删除本次追加的输出，再分别处理缺失分号和下一条语句。

4. 统一错误收集并保存根节点

入口程序在完成解析后取得根节点，遍历树中的错误标记，并与词法错误按行号统一排序。词法阶段记录单独的 `&`、`|` 后恢复其类别，使错误单词能够继续参与解析，避免只记录错误却停在原位置。

大模型辅助代码评价与人工调整

大模型辅助用于理解递归下降、语法树组织、输出顺序，以及定位错误恢复中的问题。对辅助建议结合完整文法和本次作业的错误范围进行核对，并按自身设计调整模块划分。字符串初值的补充以完整文法为依据；试探解析的撤销则同时考虑读取位置、树结构和输出记录，避免只修改其中一项。